PRAKTIKUM 3
STRUKTUR DATA

NAMA: Ihsanta Zaki Sanjaya

KELAS: 1 D4 Teknik Komputer B

NRP: 3223600058

DOSEN: Dr Bima Sena Bayu Dewantara S.ST, M.T

MATA KULIAH: Praktikum Pemograman Dasar 2

TANGGAL: Rabu, 20 Maret 2024

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2024

Praktikum 3
Struktur Data

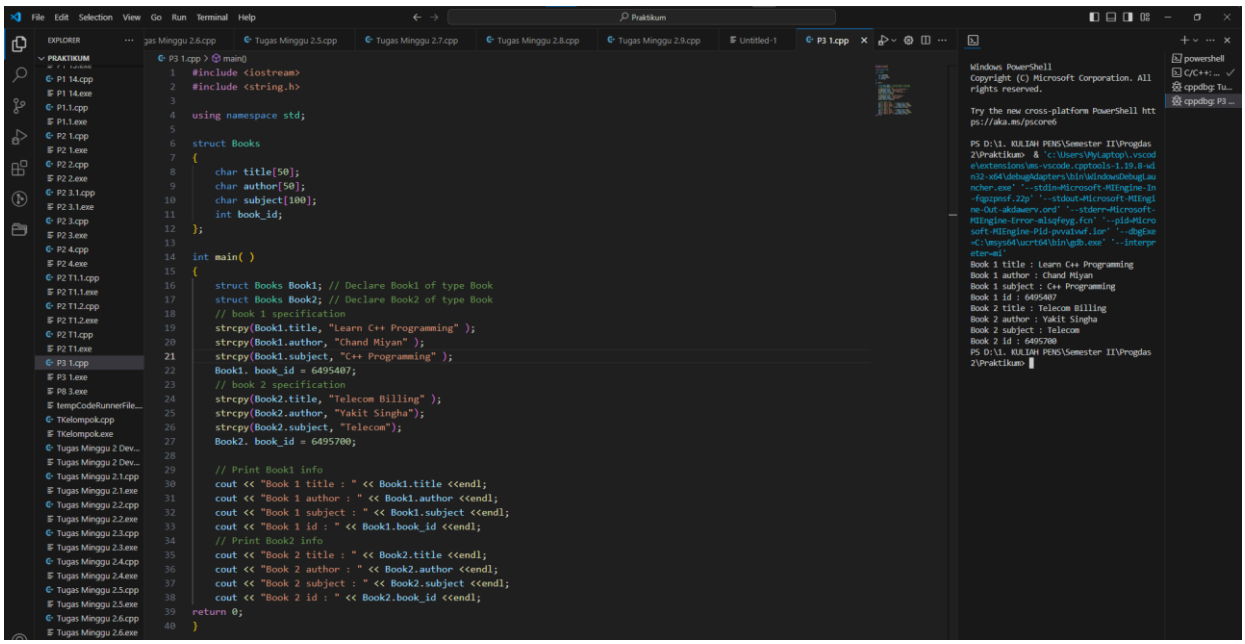
Untuk memberikan pemahaman kepada anda tentang konsep dasar struktur data dan cara menggunakan dalam program C++, maka cobalah beberapa modul percobaan dibawah ini.

1. **Percobaan 1 – Mengakses anggota struktur**
   Untuk mengakses anggota struktur, kita dapat menggunakan operator akses keanggotaan (.). Operator tersebut dikodekan sebagai periode antara nama variabel struktur dengan anggota struktur yang ingin kita akses. Kita bisa menggunakan kata kunci ***struct*** untuk mendefinisikan tipe variabel struktur. Berikut adalah contoh yang menjelaskan penggunaan struktur:

```
#include <iostream>
#include <string.h>

using namespace std;

struct Books
{
        char title[50];
        char author[50];
        char subject[100];
        int book_id;
};

int main( )
{
        struct Books Book1;  // Declare Book1 of type Book
        struct Books Book2;  // Declare Book2 of type Book

        // book 1 specification
        strcpy(Book1.title, "Learn C++ Programming" );
        strcpy(Book1.author, "Chand Miyan" );
        strcpy(Book1.subject, "C++ Programming" );
        Book1. book_id = 6495407;

        // book 2 specification
        strcpy(Book2.title, "Telecom Billing" );
        strcpy(Book2.author, "Yakit Singha");
```

```
        strcpy(Book2.subject, "Telecom");
        Book2. book_id = 6495700;

        // Print Book1 info
        cout << "Book 1 title : " << Book1.title <<endl;
        cout << "Book 1 author : " << Book1.author <<endl;
        cout << "Book 1 subject : " << Book1.subject <<endl;
        cout << "Book 1 id : " << Book1.book_id <<endl;

        // Print Book2 info
        cout << "Book 2 title : " << Book2.title <<endl;
        cout << "Book 2 author : " << Book2.author <<endl;
        cout << "Book 2 subject : " << Book2.subject <<endl;
        cout << "Book 2 id : " << Book2.book_id <<endl;
        return 0;
}
```

Hasil pemograman:



Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Analisa:

Program menggunakan struktur data Books yang memiliki atribut title, author, subject, dan book_id. Di dalam fungsi main, dua buah objek dari struktur Books dibuat, yaitu Book1 dan Book2. Kemudian, atribut-atribut dari kedua objek tersebut diinisialisasi dengan menggunakan fungsi strcpy. Setelah inisialisasi, informasi dari kedua buah buku dicetak ke layar menggunakan cout.

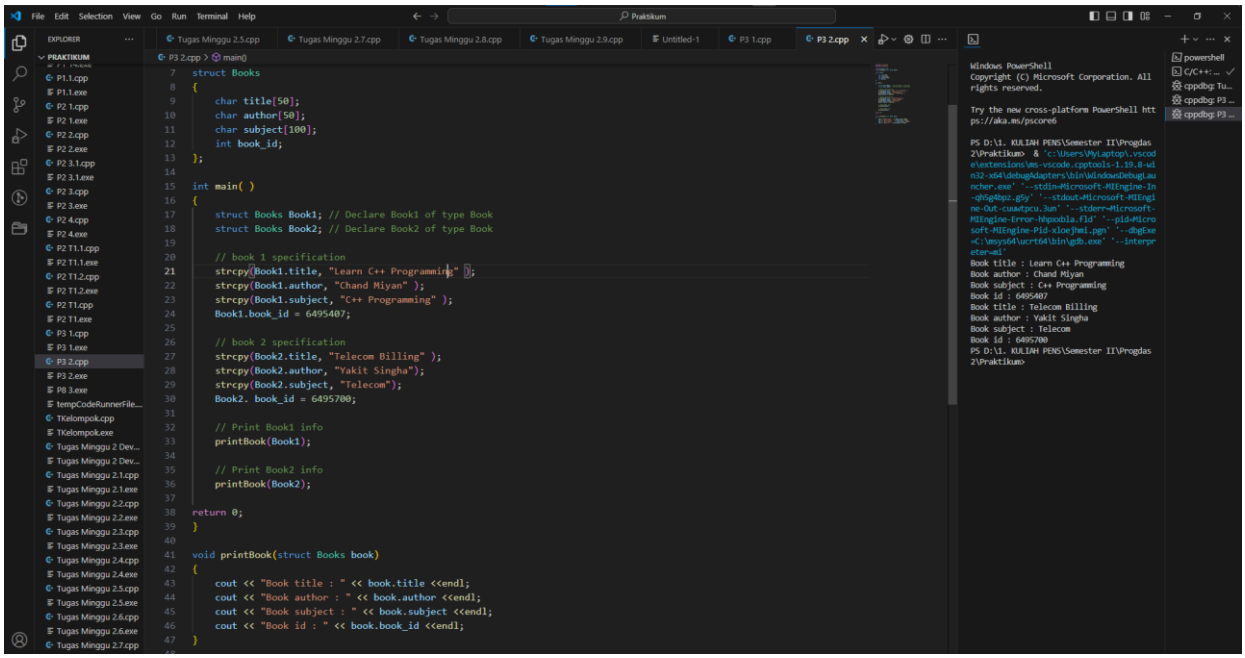
2. **Percobaan 2 – Struktur sebagai argumentasi fungsi**

   Kita juga dapat mem-passing sebuah struktur sebagai sebuah argumentasi pemanggilan fungsi sama seperti kita mem-passing variabel lain atau pointer. Kita dapat mengakses variabel struktur dengan cara yang sama seperti pada contoh 1. Pada contoh di bawah ini, pencetakan anggota struktur dilakukan pada fungsi diluar fungsi main.

```
#include <iostream>
#include <string.h>

using namespace std;

void printBook(struct Books book);

struct Books
{
	char title[50];
	char author[50];
	char subject[100];
	int book_id;
};

int main( )
{
```

```
	struct Books Book1;  // Declare Book1 of type Book
	struct Books Book2;  // Declare Book2 of type Book

	// book 1 specification
	strcpy(Book1.title, "Learn C++ Programming" );
	strcpy(Book1.author, "Chand Miyan" );
	strcpy(Book1.subject, "C++ Programming" );
	Book1.book_id = 6495407;

	// book 2 specification
	strcpy(Book2.title, "Telecom Billing" );
	strcpy(Book2.author, "Yakit Singha");
	strcpy(Book2.subject, "Telecom");
	Book2. book_id = 6495700;

	// Print Book1 info
	printBook(Book1);

	// Print Book2 info
	printBook(Book2);

	return 0;
}

void printBook(struct Books book)
{
	cout << "Book title : " << book.title <<endl;
	cout << "Book author : " << book.author <<endl;
	cout << "Book subject : " << book.subject <<endl;
	cout << "Book id : " << book.book_id <<endl;
}
```

Hasil percobaan:



Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Analisa:

Sama seperti program pertama, program ini juga menggunakan struktur data Books yang memiliki atribut yang sama dengan Program P1. Namun, dalam fungsi main, tidak langsung mencetak informasi dari kedua buku. Sebaliknya, ada fungsi printBook yang digunakan untuk mencetak informasi dari suatu objek Books yang diterima sebagai parameter. Lalu terakhir, fungsi printBook digunakan dua kali untuk mencetak informasi dari Book1 dan Book2.

3. **Percobaan 3 – Menggunakan pointer pada struktur**

   Kita dapat mendefinisikan pointer pada struktur dengan cara yang mirip dengan mendefinisikan pointer pada variabel lainnya, sebagai berikut:

   struct Books *struct_pointer;

Sekarang, kita dapat menyimpan alamat dari variabel struktur pada variabel pointer yang didefinisikan diatas. Untuk menemukan alamat dari variabel struktur, tambahkan tanda operator **'&'** sebelum nama struktur sebagai berikut:
struct_pointer = &Book1;

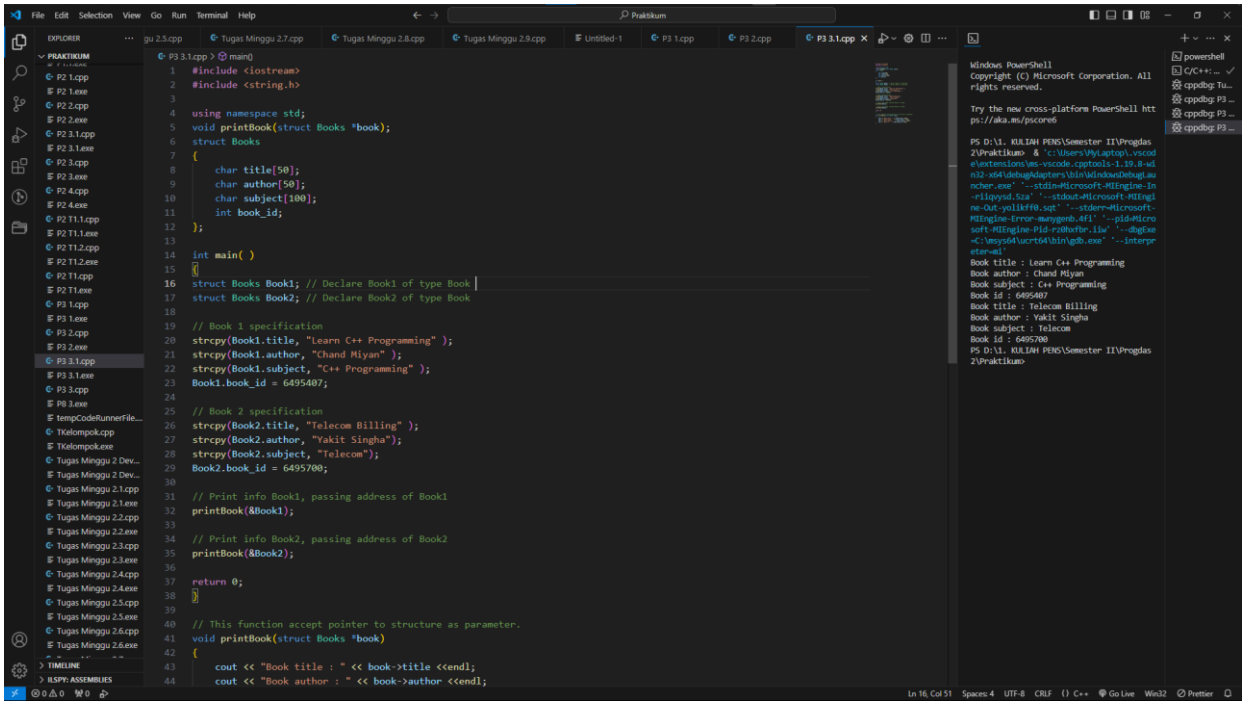
Untuk mengakses anggota struktur menggunakan sebuah pointer kepada struktur tersebut, kita harus menggunakan operator **->** sebagai berikut:
struct_pointer->title;

Untuk lebih memahami penggunaan pointer pada struktur, program dibawah ini akan membimbing anda untuk menggunakannya.

```
#include <iostream>
#include <string.h>

using namespace std;

void printBook(struct Books *book);

struct Books
{
	char title[50];
	char author[50];
	char subject[100];
	int book_id;
};

int main( )
{
	struct Books Book1;  // Declare Book1 of type Book
	struct Books Book2;  // Declare Book2 of type Book

	// Book 1 specification
	strcpy(Book1.title, "Learn C++ Programming" );
	strcpy(Book1.author, "Chand Miyan" );
	strcpy(Book1.subject, "C++ Programming" );
	Book1.book_id = 6495407;

	// Book 2 specification
	strcpy(Book2.title, "Telecom Billing" );
	strcpy(Book2.author, "Yakit Singha");
	strcpy(Book2.subject, "Telecom");
```

```
        Book2.book_id = 6495700;

        // Print Book1 info, passing address of structure
        printBook(&Book1);

        // Print Book1 info, passing address of structure
        printBook(&Book2);

        return 0;
}

// This function accept pointer to structure as parameter.
void printBook(struct Books *book)
{
        cout << "Book title : " << book->title <<endl;
        cout << "Book author : " << book->author <<endl;
        cout << "Book subject : " << book->subject <<endl;
        cout << "Book id : " << book->book_id <<endl;
}
```

Hasil percobaan:



Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Analisa:

Program menggunakan struktur data Books, namun ada perbedaan dalam penggunaan parameter fungsi printBook. Fungsi printBook menerima parameter berupa pointer ke objek Books. Dalam fungsi main, setelah inisialisasi objek Book1 dan Book2, fungsi printBook dipanggil dengan masing-masing objek sebagai argumen yang diberikan sebagai alamat memori (&Book1, &Book2).

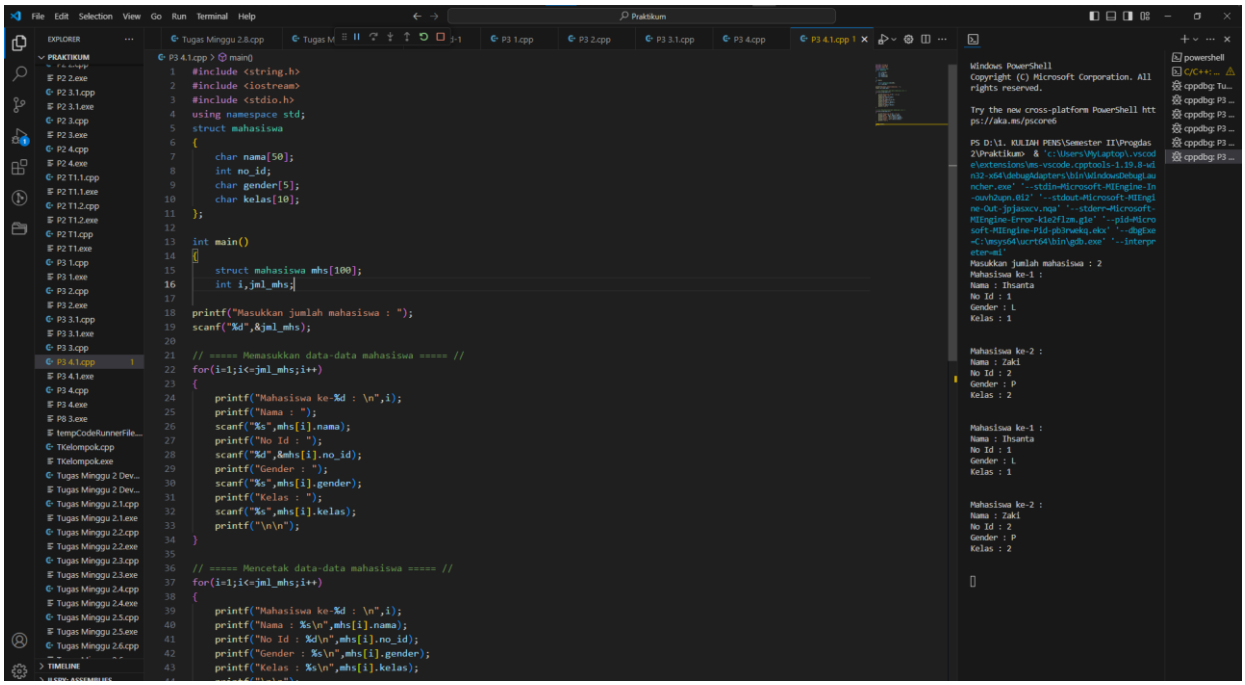
4. **Percobaan 4 – Array Struktur**

   Sama dengan variabel array lainnya, sebuah variabel struktur juga dapat didefinisikan sebagai variabel array struktur. Contoh di bawah ini menunjukkan cara menggunakan array struktur dalam pemrograman C++:

```
#include <string.h>
#include <iostream>
#include <stdio.h>

using namespace std;

struct mahasiswa
{
	char nama[50];
	int no_id;
	char gender[5];
	char kelas[10];
};

void main()
```

```
{
	struct mahasiswa mhs[100];
	int i,jml_mhs;

	printf("Masukkan jumlah mahasiswa : ");
	scanf("%d",&jml_mhs);

	// ===== Memasukkan data-data mahasiswa ===== //
	for(i=1;i<=jml_mhs;i++)
	{
		printf("Mahasiswa ke-%d : \n",i);
		printf("Nama : ");
		scanf("%s",mhs[i].nama);
		printf("No Id : ");
		scanf("%d",&mhs[i].no_id);
		printf("Gender : ");
		scanf("%s",mhs[i].gender);
		printf("Kelas : ");
		scanf("%s",mhs[i].kelas);

		printf("\n\n");
	}

	// ===== Mencetak data-data mahasiswa ===== //
	for(i=1;i<=jml_mhs;i++)
	{
		printf("Mahasiswa ke-%d : \n",i);
		printf("Nama : %s\n",mhs[i].nama);
		printf("No Id : %d\n",mhs[i].no_id);
		printf("Gender : %s\n",mhs[i].gender);
		printf("Kelas : %s\n",mhs[i].kelas);

		printf("\n\n");
	}

	_sleep(50000);
}
```

Hasil percobaan:



```
#include <string.h>
#include <iostream>
#include <stdio.h>
using namespace std;
struct mahasiswa
{
    char nama[50];
    int no_id;
    char gender[5];
    char kelas[10];
};

int main()
{
    struct mahasiswa mhs[100];
    int i,jml_mhs;

printf("Masukkan jumlah mahasiswa : ");
scanf("%d",&jml_mhs);

// ===== Memasukkan data-data mahasiswa ===== //
for(i=1;i<=jml_mhs;i++)
{
    printf("Mahasiswa ke-%d : \n",i);
    printf("Nama : ");
    scanf("%s",mhs[i].nama);
    printf("No Id : ");
    scanf("%d",&mhs[i].no_id);
    printf("Gender : ");
    scanf("%s",mhs[i].gender);
    printf("Kelas : ");
    scanf("%s",mhs[i].kelas);
    printf("\n\n");
}

// ===== Mencetak data-data mahasiswa ===== //
for(i=1;i<=jml_mhs;i++)
{
    printf("Mahasiswa ke-%d : \n",i);
    printf("Nama : %s\n",mhs[i].nama);
    printf("No Id : %d\n",mhs[i].no_id);
    printf("Gender : %s\n",mhs[i].gender);
    printf("Kelas : %s\n",mhs[i].kelas);
```

```
Masukkan jumlah mahasiswa : 2
Mahasiswa ke-1 :
Nama : Ihsanta
No Id : 1
Gender : L
Kelas : 1


Mahasiswa ke-2 :
Nama : Zaki
No Id : 2
Gender : P
Kelas : 2


Mahasiswa ke-1 :
Nama : Ihsanta
No Id : 1
Gender : L
Kelas : 1


Mahasiswa ke-2 :
Nama : Zaki
No Id : 2
Gender : P
Kelas : 2
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Analisa:

Program ini menggunakan struktur data mahasiswa yang memiliki atribut nama, no_id, gender, dan kelas. Di dalam fungsi main, pertama-tama pengguna diminta untuk memasukkan jumlah mahasiswa. Setelah itu, program meminta informasi untuk setiap mahasiswa, seperti nama, nomor ID, jenis kelamin, dan kelas. Setelah semua data dimasukkan, program mencetak informasi untuk setiap mahasiswa

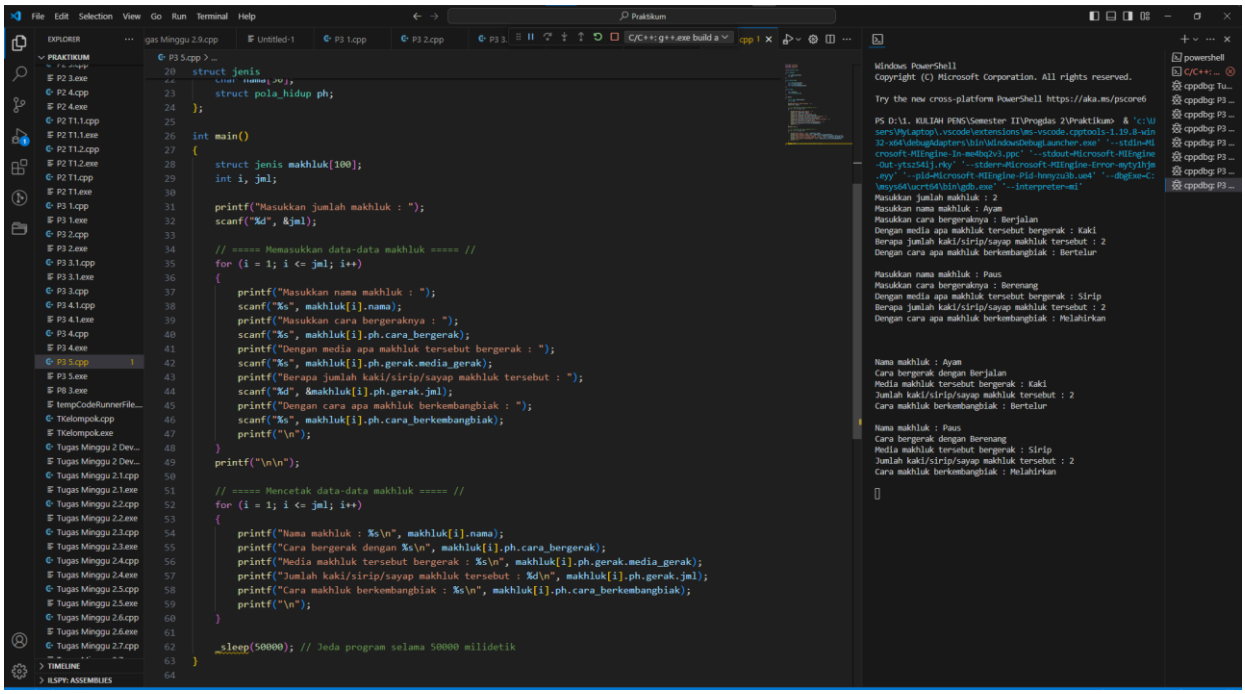
5. **Percobaan 5 – Struktur bersarang (Nested Structures)**

   Contoh program di bawah ini merupakan implementasi dari struktur bersarang, dimana di dalam sebuah struktur terdapat struktur yang lain.

```
#include <string.h>
#include <iostream>
#include <stdio.h>

using namespace std;

struct anatomi
{
	char media_gerak[50];
	int jml;
};

struct pola_hidup
{
	char cara_bergerak[50];
	struct anatomi gerak;
	char cara_berkembangbiak[50];
};

struct jenis
{
	char  nama[50];
	struct pola_hidup ph;
};

void main()
{
	struct jenis makhluk[100];
	int i,jml;

	printf("Masukkan jumlah makhluk : ");
	scanf("%d",&jml);

	// ===== Memasukkan data-data makhluk ===== //
	for(i=1;i<=jml;i++)
	{
		printf("Masukkan nama makhluk : "); scanf("%s",makhluk[i].nama);
```

```
		printf("Masukkan cara bergeraknya : ");
scanf("%s",makhluk[i].ph.cara_bergerak);
		printf("Dengan media apa makhluk tersebut bergerak : ");
scanf("%s",makhluk[i].ph.gerak.media_gerak);
		printf("Berapa jumlah kaki/sirip/sayap makhluk tersebut : ");
scanf("%d",&makhluk[i].ph.gerak.jml);
		printf("Dengan cara apa makhluk berkembangbiak : ");
scanf("%s",makhluk[i].ph.cara_berkembangbiak);

		printf("\n");
	}

	printf("\n\n");

	// ===== Mencetak data-data makhluk ===== //
	for(i=1;i<=jml;i++)
	{
		printf("Nama makhluk : %s\n",makhluk[i].nama);
		printf("Cara bergerak dengan %s\n",makhluk[i].ph.cara_bergerak);
		printf("Media makhluk tersebut bergerak :
%s\n",makhluk[i].ph.gerak.media_gerak);
		printf("Jumlah kaki/sirip/sayap makhluk tersebut :
%d\n",makhluk[i].ph.gerak.jml);
		printf("Cara makhluk berkembangbiak :
%s\n",makhluk[i].ph.cara_berkembangbiak);

		printf("\n");
	}

	_sleep(50000);
}
```

Hasil percobaan:



```
struct jenis
    struct pola_hidup ph;
};

int main()
{
    struct jenis makhluk[100];
    int i, jml;

    printf("Masukkan jumlah makhluk : ");
    scanf("%d", &jml);

    // ===== Memasukkan data-data makhluk ===== //
    for (i = 1; i <= jml; i++)
    {
        printf("Masukkan nama makhluk : ");
        scanf("%s", makhluk[i].nama);
        printf("Masukkan cara bergeraknya : ");
        scanf("%s", makhluk[i].ph.cara_bergerak);
        printf("Dengan media apa makhluk tersebut bergerak : ");
        scanf("%s", makhluk[i].ph.gerak.media_gerak);
        printf("Berapa jumlah kaki/sirip/sayap makhluk tersebut : ");
        scanf("%d", &makhluk[i].ph.gerak.jml);
        printf("Dengan cara apa makhluk berkembangbiak : ");
        scanf("%s", makhluk[i].ph.cara_berkembangbiak);
        printf("\n");
    }
    printf("\n\n");

    // ===== Mencetak data-data makhluk ===== //
    for (i = 1; i <= jml; i++)
    {
        printf("Nama makhluk : %s\n", makhluk[i].nama);
        printf("Cara bergerak dengan %s\n", makhluk[i].ph.cara_bergerak);
        printf("Media makhluk tersebut bergerak : %s\n", makhluk[i].ph.gerak.media_gerak);
        printf("Jumlah kaki/sirip/sayap makhluk tersebut : %d\n", makhluk[i].ph.gerak.jml);
        printf("Cara makhluk berkembangbiak : %s\n", makhluk[i].ph.cara_berkembangbiak);
        printf("\n");
    }

    _sleep(50000); // Jeda program selama 50000 milidetik
}
```

```
Masukkan jumlah makhluk : 2
Masukkan nama makhluk : Ayam
Masukkan cara bergeraknya : Berjalan
Dengan media apa makhluk tersebut bergerak : Kaki
Berapa jumlah kaki/sirip/sayap makhluk tersebut : 2
Dengan cara apa makhluk berkembangbiak : Bertelur

Masukkan nama makhluk : Paus
Masukkan cara bergeraknya : Berenang
Dengan media apa makhluk tersebut bergerak : Sirip
Berapa jumlah kaki/sirip/sayap makhluk tersebut : 2
Dengan cara apa makhluk berkembangbiak : Melahirkan


Nama makhluk : Ayam
Cara bergerak dengan Berjalan
Media makhluk tersebut bergerak : Kaki
Jumlah kaki/sirip/sayap makhluk tersebut : 2
Cara makhluk berkembangbiak : Bertelur

Nama makhluk : Paus
Cara bergerak dengan Berenang
Media makhluk tersebut bergerak : Sirip
Jumlah kaki/sirip/sayap makhluk tersebut : 2
Cara makhluk berkembangbiak : Melahirkan
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Analisa:

Berbeda dengan program sebelumnya, program ini menggunakan beberapa struktur data bersarang, yaitu anatomi, pola_hidup, dan jenis. Struktur anatomi memiliki atribut media_gerak dan jml, pola_hidup memiliki atribut cara_bergerak, gerak (yang bertipe anatomi), dan cara_berkembangbiak, sedangkan jenis memiliki atribut nama dan ph (yang bertipe pola_hidup). Dalam fungsi main, program meminta pengguna untuk memasukkan jumlah makhluk, lalu meminta informasi tentang setiap makhluk, termasuk nama, cara bergerak, media gerak, jumlah kaki/sirip/sayap, dan cara berkembangbiak.

Buatlah sebuah program dalam bahasa pemrograman C++ untuk melakukan operasi penjumlahan, pengurangan, dan perkalian matriks menggunakan struktur data. Struktur data digunakan untuk mendefinisikan elemen dari matriks 2 dimensi. Nama obyek dari struktur data adalah A untuk matriks pertama, B untuk matriks kedua, dan C untuk matriks hasil. Ukuran matriks ditentukan oleh pengguna dan dimasukkan pada awal program berjalan. Program ini akan melakukan operasi penjumlahan, pengurangan, dan perkalian matriks sesuai dengan nilai-nilai elemen yang dimasukkan.

Kode pemograman:

```
#include <iostream>
using namespace std;

struct Matrix {
    int size;
    int elm[100][100]; // Maksimum ukuran matriks
};

void inputMatrix(Matrix& matrix) {
    cout << "Enter elements of the " << matrix.size << "x" <<
matrix.size << " matrix:" << endl;
    for (int i = 0; i < matrix.size; ++i) {
        for (int j = 0; j < matrix.size; ++j) {
            cout << "Element [" << i << "][" << j << "]: ";
            cin >> matrix.elm[i][j];
        }
    }
}

Matrix addMatrices(const Matrix& A, const Matrix& B) { // Penjumlahan
    Matrix C;
    C.size = A.size;
    for (int i = 0; i < A.size; ++i) {
        for (int j = 0; j < A.size; ++j) {
            C.elm[i][j] = A.elm[i][j] + B.elm[i][j];
        }
    }
    return C;
}

Matrix subtractMatrices(const Matrix& A, const Matrix& B) { //
Pengurangan
    Matrix C;
    C.size = A.size;
    for (int i = 0; i < A.size; ++i) {
        for (int j = 0; j < A.size; ++j) {
```

```
            C.elm[i][j] = A.elm[i][j] - B.elm[i][j];
        }
    }
    return C;
}

Matrix multiplyMatrices(const Matrix& A, const Matrix& B) { //
Perkalian
    Matrix C;
    C.size = A.size;
    for (int i = 0; i < A.size; ++i) {
        for (int j = 0; j < A.size; ++j) {
            C.elm[i][j] = 0;
            for (int k = 0; k < A.size; ++k) {
                C.elm[i][j] += A.elm[i][k] * B.elm[k][j];
            }
        }
    }
    return C;
}

void displayMatrix(const Matrix& matrix) { // Menampilkan element
    cout << "Matrix:" << endl;
    for (int i = 0; i < matrix.size; ++i) {
        for (int j = 0; j < matrix.size; ++j) {
            cout << matrix.elm[i][j] << " ";
        }
        cout << endl;
    }
}

int main() {
    Matrix A, B, C;
    int choice;
    bool exit = false;

    while (!exit) {
        cout << "\nSelamat datang di Operasi Matriks" << endl;

        cout << "\nSilahkan pilih operasi yang ingin dilakukan:" <<
endl;
        cout << "1. Penjumlahan Matriks" << endl;
        cout << "2. Pengurangan Matriks" << endl;
        cout << "3. Perkalian Matriks" << endl;
        cout << "4. Keluar" << endl;
```

```
        cout << "\nMasukkan pilihan: ";
        cin >> choice;

        switch(choice) {
            case 1:
                cout << "Ukuran matrix A (dalam n): ";
                cin >> A.size;
                inputMatrix(A);     // Input element dari Matriks A

                cout << "Ukuran matrix B (dalam n): ";
                cin >> B.size;
                inputMatrix(B);    // Input element dari Matriks B

                C = addMatrices(A, B);      // Penjumlahan +
                cout << "\nPenjumlahan dari matrix A dan B:" << endl;
                displayMatrix(C);
                break;
            case 2:
                cout << "Ukuran matrix A (dalam n): ";
                cin >> A.size;
                inputMatrix(A);     // Input element dari Matriks A

                cout << "Ukuran matrix B (dalam n): ";
                cin >> B.size;
                inputMatrix(B);    // Input element dari Matriks B

                C = subtractMatrices(A, B);      // Pengurangan -
                cout << "\nPengurangan dari matrix A dan B:" << endl;
                displayMatrix(C);
                break;
            case 3:
                cout << "Ukuran matrix A (dalam n): ";
                cin >> A.size;
                inputMatrix(A);     // Input element dari Matriks A

                cout << "Ukuran matrix B (dalam n): ";
                cin >> B.size;
                inputMatrix(B);    // Input element dari Matriks B

                C = multiplyMatrices(A, B);    // Perkalian x
                cout << "\nPerkalian dari matrix A dan B:" << endl;
                displayMatrix(C);
                break;
            case 4:
                exit = true;
```

```
                break;
            default:
                cout << "Coba lagi!";
        }
    }

    cout << "\nTerima kasih telah menggunakan program ini." << endl;

    return 0;
}
```

Hasil pemograman:

```
D:\1. KULIAH PENS\Semester II\Progdas 2\Praktikum\Tugas Biasa.exe

Selamat datang di Operasi Matriks

Silahkan pilih operasi yang ingin dilakukan:
1. Penjumlahan Matriks
2. Pengurangan Matriks
3. Perkalian Matriks
4. Keluar

Masukkan pilihan: 1
Ukuran matrix A (dalam n): 2
Enter elements of the 2x2 matrix:
Element [0][0]: 1
Element [0][1]: 2
Element [1][0]: 3
Element [1][1]: 4
Ukuran matrix B (dalam n): 2
Enter elements of the 2x2 matrix:
Element [0][0]: 1
Element [0][1]: 2
Element [1][0]: 3
Element [1][1]: 4

Penjumlahan dari matrix A dan B:
Matrix:
2 4
6 8

Selamat datang di Operasi Matriks

Silahkan pilih operasi yang ingin dilakukan:
1. Penjumlahan Matriks
2. Pengurangan Matriks
3. Perkalian Matriks
4. Keluar

Masukkan pilihan: 2
Ukuran matrix A (dalam n): 2
Enter elements of the 2x2 matrix:
Element [0][0]: 1
Element [0][1]: 2
Element [1][0]: 3
Element [1][1]: 4
Ukuran matrix B (dalam n): 2
Enter elements of the 2x2 matrix:
Element [0][0]: 1
Element [0][1]: 2
Element [1][0]: 3
Element [1][1]: 4

Pengurangan dari matrix A dan B:
Matrix:
0 0
0 0

Selamat datang di Operasi Matriks

Silahkan pilih operasi yang ingin dilakukan:
1. Penjumlahan Matriks
2. Pengurangan Matriks
3. Perkalian Matriks
4. Keluar

Masukkan pilihan:
```

```
Selamat datang di Operasi Matriks

Silahkan pilih operasi yang ingin dilakukan:
1. Penjumlahan Matriks
2. Pengurangan Matriks
3. Perkalian Matriks
4. Keluar

Masukkan pilihan: 3
Ukuran matrix A (dalam n): 2
Enter elements of the 2x2 matrix:
Element [0][0]: 1
Element [0][1]: 2
Element [1][0]: 3
Element [1][1]: 4
Ukuran matrix B (dalam n): 2
Enter elements of the 2x2 matrix:
Element [0][0]: 1
Element [0][1]: 2
Element [1][0]: 3
Element [1][1]: 4

Perkalian dari matrix A dan B:
Matrix:
7 10
15 22
```

Kesimpulan:

1. Penggunaan Struktur Data
   Keempat program menggunakan struktur data untuk mengorganisir informasi yang terkait. Hal ini memungkinkan penyimpanan data yang terstruktur dan mempermudah akses serta manipulasi data.
2. Penggunaan Fungsi
   Program P2 dan P3 menunjukkan penggunaan fungsi untuk mencetak informasi dari objek struktur data. Penggunaan fungsi membantu dalam mengelola kode yang lebih terstruktur dan mudah dipahami.
3. Penggunaan Pointer
   Program P3 menggunakan parameter berupa pointer ke objek struktur data. Hal ini memungkinkan manipulasi data secara langsung pada objek yang ada di memori, mengurangi overhead dari pengiriman objek secara langsung ke fungsi.
4. Penggunaan Nested Structure
   Program P5 menunjukkan penggunaan struktur data bersarang (nested structure), yaitu struktur data yang memiliki struktur data lain sebagai anggota. Ini berguna untuk mengorganisir data yang memiliki hubungan yang kompleks dan terkait.